O módulo *Keyboard Reader* é constituído por três blocos principais, de acordo com o diagrama representado na Figura 1:

i) o descodificador de teclado (*Key Decode*);

ii) o bloco de armazenamento (designado por *Ring Buffer*);

iii) o bloco de entrega ao consumidor (designado por *Key Transmitter*).

Neste caso o módulo *Control*, implementado em software, é a entidade consumidora.

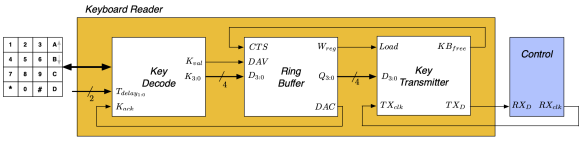
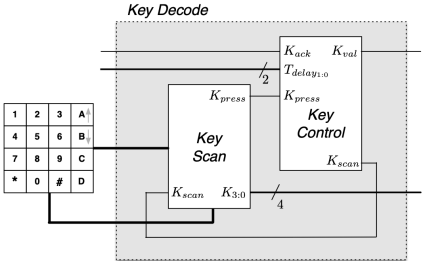
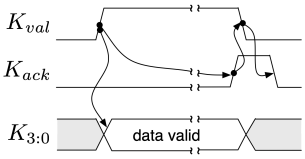


Figura 1: Diagrama de blocos do módulo *Keyboard Reader*



(a) Diagrama de blocos



(b) Diagrama temporal

Figura 2: Bloco *Key Decode*

1 Key Decode

O bloco *Key Decode* implementa um descodificador de um teclado matricial 4x4 em hardware, sendo constituído por três sub-blocos, conforme o diagrama de blocos representado na Figura 2a:

i) uma ligação ao teclado matricial 4x4, da placa de expansão fornecida;

ii) o bloco *Key Scan*, responsável pelo varrimento do teclado;

iii) o bloco *Key Control*, que realiza o controlo do varrimento e o controlo de fluxo

O controlo de fluxo de saída do bloco *Key Decode* (para o módulo *Key Buffer*), define que o sinal K_{val} é ativado quando é detetada a pressão de uma tecla, sendo também disponibilizado o código dessa tecla no *bus* $K_{3:0}$. Por ordem, do *bit* mais significativo para o menos, o código da tecla premida corresponde à coluna, seguida da linha, cada informação com 2 *bits*. Apenas é iniciado um novo ciclo de varrimento ao teclado quando o sinal K_{ack} for ativado. O diagrama temporal do controlo de fluxo está representado na Figura 2b.

1.1 Key Scan

O bloco *Key Scan* foi implementado de acordo com o diagrama de blocos representado na Figura 3.

O diagrama do bloco KeyScan da Figura 3 corresponde à 3ª versão da possível implementação, de três disponibilizadas pelos docentes.

Foi escolhida esta opção com a utilização de um *priority encoder* (*PENC*) pois este permite uma leitura mais rápida e eficiente do teclado.

Este componente permite a ativação imediata do sinal K_{press} assim que é detetada uma tecla pressionada, reduzindo a latência de resposta do sistema. Adicionalmente, assegura a priorização de linhas de maior ordem (inferiores) quando múltiplas teclas são pressionadas na mesma coluna, garantindo um comportamento determinístico.

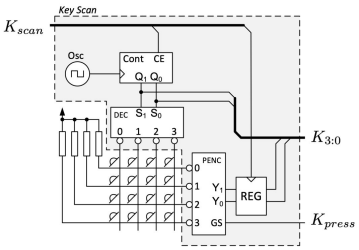


Figura 3: Diagrama de blocos do módulo *Key Scan*

1.2 Key Control

O bloco *Key Control* foi implementado pela máquina de estados representada em *ASM-chart* na Figura 4.

A nossa solução de implementação da máquina de estados, correspondente ao bloco *Key Control*, consiste na criação de 3 estados:

- *Scanning*: Ativa a saída K_{scan}, que indica que o varrimento do teclado está a ser feito, aguardando por uma tecla ser premida. Após uma tecla ser premida (K_{press} assume o valor lógico 1, correspondente a *True*), a máquina avança para o estado seguinte.

- *Sending*: Este estado envia a indicação de tecla premida, através de K_{val}. Quando a tecla premida é consumida pelo *Ring Buffer*, este envia um sinal com essa indicação, K_{ack}, que faz o controlador mudar de estado.

- *Acknowledged*: Após a tecla ser consumida pelo *Ring Buffer*, este estado aguarda, independentemente de outra tecla ser premida, a transição descendente de K_{ack} que, caso outra tecla seja premida (K_{press} com valor lógico '1'), envia a máquina para o estado *Sending*. Caso não seja premida outra tecla, com a transição descendente de K_{ack}, a máquina volta para o estado inicial *Scanning*, aguardando que outra tecla seja premida.

Esta implementação, em particular após o último estado, foi feita tendo em conta, numa fase futura, a adição de *Tdelay*, que premite o pressionamento de outra tecla após a primeira ter sido registada.

A descrição hardware do bloco *Key Decode* em *VHDL* encontra-se no Anexo A.3.

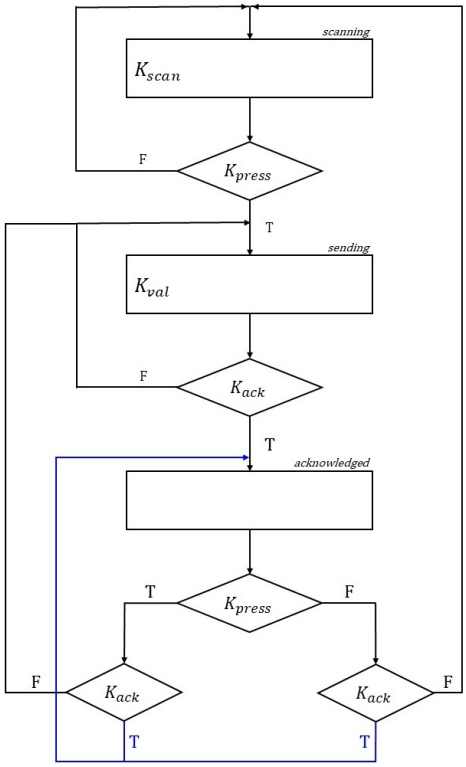


Figura 4: Máquina de estados do *Key Control*

2 Conclusões

Este componente, *Keyboard Reader*, é responsável pela leitura do teclado, na parte de *hardware*, através de um contador para varrer cada coluna e de um *Priority Encoder* para varrer cada linha.

Quando uma tecla é premida, o varrimento de linhas e colunas é parado e estas são registadas, em $K_{3:0}$. Existe também uma variável, K_{val}, que indica que existe um valor a ser registado. Estes valores são, atualmente, enviados para o *USBPort*, que o transmite do *hardware* para o *software*.

No *software*, através do objeto *KBD*, é lida a posição, linha e coluna, passada pelo *hardware* e, através de uma matriz definida no objeto, é devolvida a informação, pela função *getKey()*, da tecla que foi premida.

Referências

References

[1] Laboratório de Informática e Computadores - Slides de apoio às aulas, 2025/2026.

[2] Laboratório de Informática e Computadores - Enunciado do Projeto "Ticket Machine", 2025/2026.

[3] Pedro Miguens Matutino, Sérgio André *Guia de Instalação do Quartus Prime Lite Edition 20.1.1.720*, 2022, ISEL.

A VHDL

A.1 Keyboard Reader

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyboardReader is
        port(
                Clk     : in  std_logic;
                Reset   : in  std_logic;
                Kack    : in  std_logic;
                KbdLin  : in  std_logic_vector(3 downto 0);
                Tdelay  : in  std_logic_vector(1 downto 0);
                KbdCol  : out std_logic_vector(3 downto 0);
                TXd     : out std_logic;
                UPort   : out std_logic_vector(7 downto 0)
        );
end KeyboardReader;

architecture structural of KeyboardReader is

component KeyDecode is
        port(
                Clk, Reset, Kack    : in  std_logic;
                Tdelay              : in  std_logic_vector(1 downto 0);
                KbdLin              : in  std_logic_vector(3 downto 0);
                Kval                : out std_logic;
                Kcode               : out std_logic_vector(3 downto 0);
                KbdCol              : out std_logic_vector(4 downto 1)
        );
end component KeyDecode;

signal Kval:        std_logic;
signal K:                std_logic_vector(3 downto 0);

begin
        Decode:        KeyDecode port map(Clk => Clk, Reset => Reset, Kack => Kack, Tdelay => Tdelay,
                KbdLin => KbdLin, Kval => Kval, Kcode => K, KbdCol => KbdCol);
        UPort <= "000" & Kval & K;
end structural;
```

A.2 Keyboard Reader testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyboardReader_tb is
end entity;

architecture KeyboardReader_tb_arch of KeyboardReader_tb is

component KeyboardReader
        port(
                Clk, Reset, Kack    : in  std_logic;
                KbdLin              : in  std_logic_vector(3 downto 0);
                Tdelay              : in  std_logic_vector(1 downto 0);
                KbdCol              : out std_logic_vector(3 downto 0);
                TXd                 : out std_logic;
                UPort               : out std_logic_vector(7 downto 0)
        );
end component;

signal MClk_TB                      : std_logic := '1';
signal Reset_TB, Kack_TB, TXd_TB    : std_logic;
signal KbdLin_TB, KbdCol_TB         : std_logic_vector(3 downto 0);
signal Tdelay_TB                    : std_logic_vector(1 downto 0);
signal UPort_TB                     : std_logic_vector(7 downto 0);
constant MClk_period                : time := 10000 ps;
constant MClk_half_period           : time := MClk_period / 2;

begin
        MClk_TB <= not(MClk_TB) after MClk_half_period;
        UUT: KeyboardReader port map(
                Clk    => MClk_TB,
                Reset  => Reset_TB,
                Kack   => Kack_TB,
                KbdLin => KbdLin_TB,
                Tdelay => Tdelay_TB,
                KbdCol => KbdCol_TB,
                TXd    => TXd_TB,
                UPort  => UPort_TB
        );
stimulus: process
begin
        Reset_TB  <= '1';
        Kack_TB   <= '0';
        KbdLin_TB <= "1111";
        wait for MClk_period;
        Reset_TB  <= '0';
        wait for MClk_period * 4;
        KbdLin_TB <= "1011";
        wait for MClk_period;
        Kack_TB   <= '1';
        wait for MClk_period;
        Kack_TB   <= '0';
        KbdLin_TB <= "1111";
        wait for MClk_period * 4;
end process;
end architecture;
```

A.3 Key Decode

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyDecode is
        port(
                Clk     : in  std_logic;
                Reset   : in  std_logic;
                KbdLin  : in  std_logic_vector(3 downto 0);
                Tdelay  : in  std_logic_vector(1 downto 0);
                Kack    : in  std_logic;
                KbdCol  : out std_logic_vector(3 downto 0);
                Kval    : out std_logic;
                Kcode   : out std_logic_vector(3 downto 0)
        );
end KeyDecode;

architecture structural of KeyDecode is

component KeyScan is
        port(
                Clk     : in  std_logic;
                rst     : in  std_logic;
                Kscan   : in  std_logic;
                KbdLin  : in  std_logic_vector(3 downto 0);
                KbdCol  : out std_logic_vector(3 downto 0);
                K       : out std_logic_vector(3 downto 0);
                Kpress  : out std_logic
        );
end component KeyScan;

component KeyControl is
        port(
                Kack, Kpress, rst, clk  : in std_logic;
                Kval, Kscan             : out std_logic
        );
end component KeyControl;

signal Press, Scan  : std_logic;
signal ntClk        : std_logic;

begin
        ntClk <= not(Clk);
        Ctrl  : KeyControl port map(Kack => Kack, Kpress => Press, rst => Reset, clk => ntClk,
                Kval => Kval, Kscan => Scan);
        Scann : KeyScan    port map(Clk => Clk, rst => Reset, Kscan => Scan, KbdLin => KbdLin,
                KbdCol => KbdCol, K => Kcode, Kpress => Press);
end structural;
```

A.4 Key Decode testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyDecode_tb is
end entity;

architecture KeyDecode_tb_arch of KeyDecode_tb is

component KeyDecode
        port(
                Clk     : in  std_logic;
                Reset   : in  std_logic;
                Kack    : in  std_logic;
                Tdelay  : in  std_logic_vector(1 downto 0);
                KbdLin  : in  std_logic_vector(3 downto 0);
                Kval    : out std_logic;
                KbdCol  : out std_logic_vector(3 downto 0);
                Kcode   : out std_logic_vector(3 downto 0)
        );
end component;

signal Clk_TB                       : std_logic := '1';
signal Reset_TB, Kack_TB, Kval_TB   : std_logic;
signal Tdelay_TB                    : std_logic_vector(1 downto 0);
signal KbdLin_TB                    : std_logic_vector(3 downto 0) := "1111";
signal KbdCol_TB, Kcode_TB          : std_logic_vector(3 downto 0);
constant Clk_period                 : time := 10000 ps;
constant Clk_half_period            : time := Clk_period / 2;

begin
        Clk_TB <= not(Clk_TB) after Clk_half_period;
        UUT: KeyDecode port map(
                Clk    => Clk_TB,
                Reset  => Reset_TB,
                Kack   => Kack_TB,
                Tdelay => Tdelay_TB,
                KbdLin => KbdLin_TB,
                Kval   => Kval_TB,
                KbdCol => KbdCol_TB,
                Kcode  => Kcode_TB
        );
stimulus: process
begin
        Kack_TB    <= '0';
        Tdelay_TB  <= "00";
        Reset_TB   <= '1';
        wait for Clk_period;
        Reset_TB   <= '0';
        wait for Clk_period * 4;
        KbdLin_TB  <= "1101";
        wait for Clk_period * 2;
        Kack_TB    <= '1';
        KbdLin_TB  <= "1111";
        wait for Clk_period * 4;
end process;
end architecture;
```

A.5 Key Scan

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyScan is
        port(
                Clk     : in  std_logic;
                rst     : in  std_logic;
                Kscan   : in  std_logic;
                KbdLin  : in  std_logic_vector(3 downto 0);
                KbdCol  : out std_logic_vector(3 downto 0);
                K       : out std_logic_vector(3 downto 0);
                Kpress  : out std_logic
        );
end KeyScan;

architecture structural of KeyScan is

component Counter is
        port(
                CE,Reset    : in  std_logic;
                MClk        : in  std_logic;
                Q           : out std_logic_vector(1 downto 0)
        );
end component Counter;

component Decoder is
        port(
                S:        in  std_logic_vector(1 downto 0);
                E:        in  std_logic;
                O:  out std_logic_vector(3 downto 0)
        );
end component Decoder;

signal Q,Ys,RgO         : std_logic_vector(1 downto 0);
signal nKbdLin, DecO    : std_logic_vector(3 downto 0);
signal REGclk           : std_logic;

        begin
                Cont:        Counter                        port map(CE => Kscan, Reset => rst, MClk => Clk, Q => Q);
                DEC:         Decoder port map(S(0) => Q(0), S(1) => Q(1), E => '1', O => DecO);
                KbdCol <= not DecO;
                nKbdLin <= not KbdLin;
                REGclk <= not Kscan;
                PEnc: entity work.PEnc42 port map (I=>nKbdLin,A=>Ys, Gs=>Kpress );
                RG: entity work.RG2 port map(D=>Ys , Q=>RgO ,En=>'1',CLK=>REGclk, Reset=>rst);
--                MUX:         MUX4_1KeyScan        port map(I0 => KbdLin(0), I1 => KbdLin(1), I2 => KbdLin(2),
--                        I3 => KbdLin(3), S0 => Q(0), S1 => Q(1), Y => Kpress);
                K <= Q & RgO;

end structural;
```

A.6 Key Scan testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyScan_tb is
end entity;

architecture KeyScan_tb_arch of KeyScan_tb is

component KeyScan
        port(
                Clk, rst, Kscan : in  std_logic;
                KbdLin          : in  std_logic_vector(3 downto 0);
                KbdCol          : out std_logic_vector(3 downto 0);
                K               : out std_logic_vector(3 downto 0);
                Kpress          : out std_logic
        );
end component;

signal Clk_TB                       : std_logic := '1';
signal Reset_TB, Kscan_TB, Kpress_TB : std_logic;
signal KbdLin_TB, KbdCol_TB, K_TB         : std_logic_vector(3 downto 0);
constant Clk_period                 : time := 10000 ps;
constant Clk_half_period            : time := Clk_period / 2;

begin
        Clk_TB <= not(Clk_TB) after Clk_half_period;
        UUT: KeyScan port map(
                Clk    => Clk_TB,
                rst    => Reset_TB,
                Kscan  => Kscan_TB,
                KbdLin => KbdLin_TB,
                KbdCol => KbdCol_TB,
                K      => K_TB,
                Kpress => Kpress_TB
        );
stimulus: process
begin
        Kscan_TB   <= '1';
        KbdLin_TB  <= "1111";
        Reset_TB   <= '1';
        wait for Clk_period;
        Reset_TB   <= '0';
        wait for Clk_period * 4;
        KbdLin_TB  <= "1011";
        Kscan_TB   <= '0';
        wait for Clk_period;
        KbdLin_TB  <= "1111";
        Kscan_TB   <= '1';
        wait for Clk_period * 4;
end process;
end architecture;
```

A.7 Counter

```
library ieee;
use ieee.std_logic_1164.all;

entity Counter is port(
        CE,Reset     : in  std_logic;
        MClk         : in  std_logic;
        Q            : out std_logic_vector(1 downto 0)
);
end Counter;

architecture structural of Counter is
        signal CurrQ,RGtoAdd : std_logic_vector(1 downto 0);

begin
        Adder : entity work.Adder port map(A => RGtoAdd, B=>"00", Cin=>'1', R=>CurrQ);
        Reg   : entity work.RG2   port map(CLK =>MCLK, D=>CurrQ, Q=>RGtoAdd, En=>CE, Reset=>Reset);
        Q <= RGtoAdd;
end structural;
```

A.8 Counter testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity Counter_tb is
end entity;

architecture Counter_tb_arch of Counter_tb is

component Counter
        port(
                CE,Reset, MClk  : in std_logic;
                Q               : out std_logic_vector(1 downto 0)
        );
end component;

signal CE_TB, Reset_TB     : std_logic;
signal MClk_TB             : std_logic := '1';
signal Q_TB                : std_logic_vector(1 downto 0);
constant MClk_period       : time := 10000 ps;
constant MClk_half_period  : time := MClk_period / 2;

begin
        MClk_TB <= not(MClk_TB) after MClk_half_period;
        UUT: Counter port map(
                CE    => CE_TB,
                Reset => Reset_TB,
                MClk  => MClk_TB,
                Q     => Q_TB
        );
stimulus: process
begin
        CE_TB     <= '0';
        Reset_TB  <= '1';
        wait for MClk_period;
        Reset_TB  <= '0';
        CE_TB     <= '1';
        wait for MClk_period * 4;
end process;
end architecture;
```

A.9 Adder

```
library ieee;
use ieee.std_logic_1164.all;

entity Adder is
        port(
                A, B    : in  std_logic_vector(1 downto 0);
                Cin     : in  std_logic;
                R       : out std_logic_vector(1 downto 0);
                Cout    : out std_logic
        );
end Adder;

architecture structural of Adder is

signal carry: std_logic;

begin
        U1 : entity work.FA port map(A => A(0), B => B(0), Ci => Cin  , S => R(0), Co => carry);
        U2 : entity work.FA port map(A => A(1), B => B(1), Ci => carry, S => R(1), Co => Cout);
end structural;
```

A.10 FA

```
library ieee;
use ieee.std_logic_1164.all;

entity FA is
        port(
                A, B, Ci  : in  std_logic;
                S, Co     : out std_logic
        );
end FA;

architecture structural of FA is

signal sum   : std_logic;
signal carry : std_logic_vector(1 downto 0);

begin
        U1 : entity work.HA port map(A => A  , B => B , S => sum, Co => carry(0));
        U2 : entity work.HA port map(A => sum, B => Ci, S => S  , Co => carry(1));
        Co <= carry(0) or carry(1);
end structural;
```

A.11 HA

```
library ieee;
use ieee.std_logic_1164.all;

entity HA is
        port(
                A, B  : in  std_logic;
                S, Co : out std_logic
        );
end HA;

architecture structural of HA is
begin
        S  <= A xor B;
        Co <= A and B;
end structural;
```

A.12 RG2

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY RG2 IS
        PORT(
                D               : in  std_logic_vector(1 downto 0);
                CLK,EN,Reset    : in  STD_LOGIC
                Q               : out STD_LOGIC_vector(1 downto 0);
        );
END RG2;

ARCHITECTURE logicFunction OF RG2 IS
BEGIN
        FDD0: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset, D=>D(0), Q=>Q(0), SET =>'0', EN=>EN);
        FDD1: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset, D=>D(1), Q=>Q(1), SET =>'0', EN=>EN);
END LogicFunction;
```

A.13 Decoder

```
library ieee;
use ieee.std_logic_1164.all;

entity Decoder is
        port(
                S:  in  std_logic_vector(1 downto 0);
                E:  in  std_logic;
                O:  out std_logic_vector(3 downto 0)
        );
end Decoder;

architecture structural of Decoder is

component DeMUX is
        port(
                Y: in  std_logic;
                E: in  std_logic;
                S: in  std_logic;
                O: out std_logic_vector(1 downto 0)
        );
end component DeMUX;

signal Outs : std_logic_vector(1 downto 0);

begin
        DeMUX1: DeMUX port map(Y => '1'    , E => E, S => S(1), O    => Outs);
        DeMUX2: DeMUX port map(Y => Outs(0), E => E, S => S(0), O(0) => O(0), O(1) => O(1));
        DeMUX3: DeMUX port map(Y => Outs(1), E => E, S => S(0), O(0) => O(2), O(1) => O(3));
end structural;
```

A.14 Decoder testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity Decoder_tb is
end entity;

architecture Decoder_tb_arch of Decoder_tb is

component Decoder
        port(
                S:        in  std_logic_vector(1 downto 0);
                E:        in  std_logic;
                O:         out std_logic_vector(3 downto 0)
        );
end component;

signal E_TB, Clk_TB         : std_logic := '1';
signal S_TB                 : std_logic_vector(1 downto 0);
signal O_TB                 : std_logic_vector(3 downto 0);
constant Clk_period         : time := 10000 ps;
constant Clk_half_period    : time := Clk_period / 2;

begin
        Clk_TB <= not(Clk_TB) after Clk_half_period;
        UUT: Decoder port map(
                S => S_TB,
                E => E_TB,
                O => O_TB
        );
stimulus: process
begin
        E_TB <= '1';
        S_TB <= "00";
        wait for Clk_period;
        S_TB <= "01";
        wait for Clk_period;
        S_TB <= "10";
        wait for Clk_period;
        S_TB <= "11";
        wait for Clk_period;
end process;
end architecture;
```

A.15 DeMUX

```
library ieee;
use ieee.std_logic_1164.all;


entity DeMUX is
        port(
                Y: in  std_logic;
                E: in  std_logic;
                S: in  std_logic;
                O: out std_logic_vector(1 downto 0)
        );
end DeMUX;

architecture structural of DeMUX is
        begin
                O(0) <= E and not(S) and Y;
                O(1) <= E and S and Y;
end structural;
```

A.16 PEnc 4:2

```
library ieee;
use ieee.std_logic_1164.all;

entity Decoder is
        port(
                S:  in  std_logic_vector(1 downto 0);
                E:  in  std_logic;
                O:  out std_logic_vector(3 downto 0)
        );
end Decoder;

architecture structural of Decoder is

component DeMUX is
        port(
                Y: in  std_logic;
                E: in  std_logic;
                S: in  std_logic;
                O: out std_logic_vector(1 downto 0)
        );
end component DeMUX;

signal Outs : std_logic_vector(1 downto 0);

begin
        DeMUX1: DeMUX port map(Y => '1'    , E => E, S => S(1), O    => Outs);
        DeMUX2: DeMUX port map(Y => Outs(0), E => E, S => S(0), O(0) => O(0), O(1) => O(1));
        DeMUX3: DeMUX port map(Y => Outs(1), E => E, S => S(0), O(0) => O(2), O(1) => O(3));
end structural;
```

A.17 PEnc

```
library ieee;
use ieee.std_logic_1164.all;

entity PEnc is
        port(
                I       : in std_logic_vector(1 downto 0);
                A,Gs    : out std_logic
        );
end PEnc;

architecture structural of PEnc is
begin
        Gs <= I(0) or I(1);
        A  <= not I(0);
end structural;
```

A.18 Key Control

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyControl is
        port(
                Kack, Kpress, rst, clk  : in  std_logic;
                Kval, Kscan             : out std_logic
        );
end KeyControl;

architecture behavioral of KeyControl is

type STATES is (SCANNING, SENDING, ACKNOWLEDGED);
signal CurrentState, NextState: STATES;

begin
        CurrentState <= SCANNING when rst = '1' else NextState when rising_edge(Clk);
        Kscan        <= '1' when (CurrentState = SCANNING) else '0';
        Kval         <= '1' when (CurrentState = SENDING) else '0';

        GenerateNextState:
        process(CurrentState, Kack, Kpress)
        begin
                case CurrentState is
                        when SCANNING        => if(Kpress = '1') then NextState <= SENDING;
                        else NextState <= SCANNING;
                        end if;

                        when SENDING => if(Kack = '1') then NextState <= ACKNOWLEDGED;
                        else NextState <= SENDING;
                        end if;

                        when ACKNOWLEDGED => if(Kpress = '0' and Kack = '0') then NextState <= SCANNING;
                        elsif(Kpress = '1' and Kack = '0') then NextState <= SENDING;
                        else NextState <= ACKNOWLEDGED;
                        end if;
                end case;
        end process;
end behavioral;
```

A.19 Key Control testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyControl_tb is
end entity;

architecture KeyControl_tb_arch of KeyControl_tb is

component KeyControl
        port(
                Kack, Kpress, rst, clk  : in  std_logic;
                Kval, Kscan             : out std_logic
        );
end component;

signal Kack_TB, Kpress_TB, rst_TB   : std_logic;
signal Clk_TB                       : std_logic := '1';
signal Kval_TB, Kscan_TB            : std_logic;
constant Clk_period                 : time := 10000 ps;
constant Clk_half_period            : time := Clk_period / 2;

begin
        Clk_TB <= not(Clk_TB) after Clk_half_period;
        UUT: KeyControl port map(
                Kack   => Kack_TB,
                Kpress => Kpress_TB,
                rst    => rst_TB,
                clk    => Clk_TB,
                Kval   => Kval_TB,
                Kscan  => Kscan_TB
        );
stimulus: process
begin
        Kack_TB    <= '0';
        Kpress_TB  <= '0';
        rst_TB     <= '1';
        wait for Clk_period;
        rst_TB     <= '0';
        wait for Clk_period * 2;
        Kack_TB    <= '0';
        Kpress_TB  <= '1';
        wait for Clk_period * 2;
        Kack_TB    <= '1';
        Kpress_TB  <= '1';
        wait for Clk_period * 2;
        Kack_TB    <= '1';
        Kpress_TB  <= '0';
        wait for Clk_period * 2;
        Kack_TB    <= '0';
        Kpress_TB  <= '1';
        wait for Clk_period * 2;
end process;
end architecture;
```

A.20 FFD

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY FFD IS
        PORT(
                CLK   : in std_logic;
                RESET : in STD_LOGIC;
                SET   : in std_logic;
                D     : IN STD_LOGIC;
                EN    : IN STD_LOGIC;
                Q     : out std_logic
        );
END FFD;

ARCHITECTURE logicFunction OF FFD IS
BEGIN
        Q <= '0' when RESET = '1' else '1' when SET = '1' else D WHEN rising_edge(clk) and EN = '1';
END LogicFunction;
```

B *Kotlin*

B.1 HAL

Algoritmo 1: HAL - Hardware Abstract Layer

```
object HAL {
    var outs = 0b00000000
    // Inicia o objeto
    fun init(){
        clrBits(0b11111111)
    }

    // Retorna 'true' se o bit definido pela mask esta com o valor logico '1' no UsbPort
    fun isBit(value: Int): Boolean {
        val currBits = UsbPort.read()
        return (value and currBits) != 0
    }

    // Retorna os valores dos bits representados por mask presentes no UsbPort
    fun readBit(mask:Int): Int{
        val currBits = UsbPort.read()
        return (mask and currBits)
    }

    // Escreve nos bits representados por mask os valores dos bits correspondentes em value
    fun writeBit(mask:Int, value: Int) {
        outs = (outs and mask.inv()) or (value and mask)
        UsbPort.write(outs)
    }

    // Coloca os bits representados por mask no valor logico '1'
    fun setBits(mask:Int) {
        outs = outs or mask
        UsbPort.write(outs)
    }

    // Coloca os bits representados por mask no valor logico '0'
    fun clrBits(mask:Int) {
        outs = outs and mask.inv()
        UsbPort.write(outs)
    }
}
```

B.2 KBD

Algoritmo 2: KBD

```
// Ler teclas. Funcoes retornam '0'..'9','A'..'D','#','*' ou NONE.
object KBD {
    const val NONE = 0;
    val matrix = arrayOf(
        arrayOf('1', '2', '3', 'A'),
        arrayOf('4', '5', '6', 'B'),
        arrayOf('7', '8', '9', 'C'),
        arrayOf('*', '0', '#', 'D')
    )

    // Inicia a classe
    fun init() {}

    // Retorna de imediato a tecla premida ou NONE se nao ha tecla premida.
    fun getKey(): Char {
        val keyCol = HAL.readBit(0b00001100) shr 2
        val keyLin = HAL.readBit(0b00000011)
        val Kval = HAL.isBit(kValMask)
        return if(Kval) {
            matrix[keyLin][keyCol]
        } else NONE.digitToChar()
    }
```

```
    // Retorna a tecla premida, caso ocorra antes do 'timeout' (em milissegundos),
    // ou NONE caso contrario.
    fun waitKey(timeout: Long): Char {
        val startTime = Time.getTimeInMillis()
        while(Time.getTimeInMillis() - startTime < timeout) {
            val key = getKey()
            if(key != NONE.digitToChar()) { return key }
        }
        return NONE.digitToChar()
    }

}
```